Ao3 Metadata Mapper Test Plan

(Adapted from: IMT 542 Assignment G9: Control for quality and performance of information system)

Em Stelter, 6/5/2026

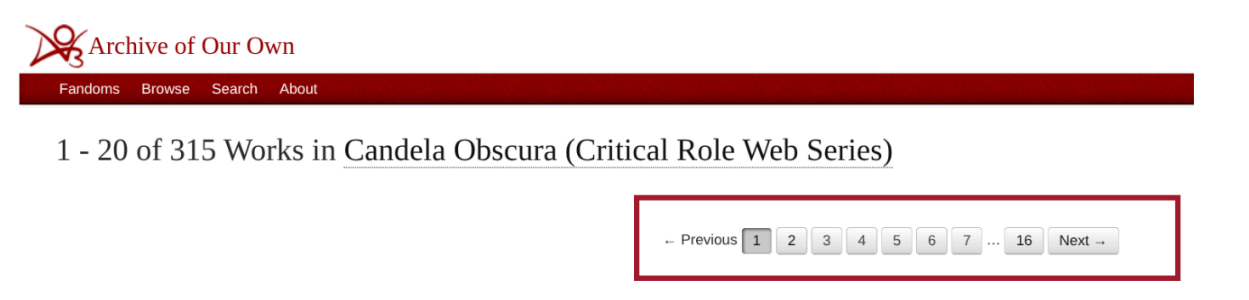
Purpose

This document outlines the recommended testing strategy for Ao3MM users.

System Overview

Ao3 Metadata Mapper organizes fanwork metadata in a JSON array. Ao3MM generates the JSON array in the following steps:

1. First, it takes in the first webpage of an Ao3 fandom (example: https://archiveofourown.org/tags/Candela%20Obscura%20(Critical%20Role%20Web%20Series)/works) and scrapes the page for a list of "pagination links" (box circled in red below). The combined pagination links contain links to all the fanworks for the given Ao3 fandom, but each pagination link only displays 20 fanworks. I have not yet found a reliable way in Ao3's UI to display all fanwork links on a single page, so it's necessary to scrape for a list of individual pagination links.



2. Second, for each pagination link, Ao3MM scrapes the webpage for a list of links to the 20 fanworks on that page.
3. Third, for each fanwork link, Ao3MM scrapes the page for the desired metadata, then organizes the metadata into a dictionary. Ao3MM then compiles the individual fanwork dictionaries into a nested dictionary.

4. Fourth, Ao3MM converts the nested dictionary of all fanwork metadata into a JSON array and saves the JSON file to local storage for safekeeping.
5. Last, Ao3MM loads the saved JSON file, converts it to a dataframe, and uses plotly.express to generate visualizations of the saved metadata.

Test Objectives

- Ensure that web scraping processes capture:
    - All pagination links for the given Ao3 fandom
    - All individual fanwork links for each pagination link
    - Complete html documents for each fanwork link scraped
- Ensure that text processing effectively parses and extracts complete, desired metadata from fanworks
- Ensure that extracted metadata is effectively compiled & converted to JSON array and saved locally
- Ensure that multiple JSON arrays are successfully merged into a single nested JSON array

Functional Testing

These are the functional tests I've developed as I have been writing and debugging my code, and these are the tests I would expect another hobbyist fan with a similar level of Python skill to run when using Ao3MM to build their own visualizations. If I were to implement this as a product for market / on a larger scale, I would incorporate more automation.

However, the expectation here is that individuals will use this code to generate fanwork visualizations on a one-off basis, or perhaps a semi-regular cadence, but Ao3MM is not so much intended for real-time monitoring or daily use as it is for individual meta analysis projects.

Test Case	Description / Method	Expected Result
get pagination links	run step1_test_pagination.py This script will read in the first pagination link for a sample Ao3	The script will return a list of pagination links for the entire fandom (i.e. a list of links to every page of search results for the

	fandom (i.e. the first page of search results for the sample fandom's Ao3 tag) and scrape the test link.	sample fandom's Ao3 tag), then save that list as a .txt file. For the current sample fandom selected, "Candela Obscura (Critical Role Web Series)", the script should return a .txt file with 11 pagination links (as of 6/5/2026).
get_fanwork_links	Run step2_test_fanwork_links.py This script will read in a list of pagination links from the file 'pagination_links.txt', select a test pagination link from the list, and scrape the test pagination link.	The script will return a list of fanwork links for the test pagination link, then save that list as a .txt file. There should be 20 fanwork links in the .txt file, and none of the fanwork links should contain 'chapter' in the link string (i.e. none of the links should go to specific chapters; each link should just go to the first page of the fanwork).
parse_html_doc: individual fanwork links AND parse_html_doc: fanwork links list	Run step3_test_parse_html.py This script will run three tests. The first two tests scrape metadata and create a JSON array for individual fanwork links, one link that contains adult content (i.e. is rated "Mature" or "Explicit") and one that does not (i.e. is rated "Not Rated," "General Audiences, or "Teen And Up Audiences". The third test scrapes metadata and creates a nested JSON array for the first 10 fanwork links in the text file: "fanwork_links.txt".	For the first two (individual link) tests, the script will return and save two respective JSON files: test_fanwork_json1.json and test_fanwork_json2.json. Both resulting JSON files should have populated metadata fields (1-2 fields per file may have value='Unknown' due to not-yet-solved bugs in the web scraping code, but no more than 1-2 fields.) For the third (link list) test, the script will return a nested JSON file: 'first10works_test.json'. There should be a total of 10 JSON arrays with fanwork data nested within the file. 1-2 fields per fanwork may have

		value='Unknown' due to not-yet-solved bugs in the web scraping code, but no more than 1-2 fields.)
test_merge_JSON	Run step4_test_merge_JSON.py	This test will take a list of JSON files created from fandom pagination links and merge them into a new nested JSON structure, then save that nested structure as 'all_candela_test.json'.

Performance Testing

These performance tests are not yet fully developed, so specific scripts and instructions are not available like they are for the Functional Testing section. However, these would help respond to some of the challenges I encountered with RAM / memory issues when trying to scrape too many webpages in a single script or notebook cell. If I were to operationalize Ao3MM on a larger scale, these performance tests would be how I would determine the efficacy of the tool for collecting metadata from larger Ao3 fandoms (i.e. fandoms with more fanworks) and work to increase the tool's efficacy.

Test Case	Description / Method	Expected Result
20 fanwork links (OR: single pagination link)	Input a single pagination link (i.e. link of search results for any given fandom tag). **Run methods from:** Step2_get_fanwork_links.py, Step3_parse_html_docs.py, and Step4_merge_json_files.py A single pagination link, if scraped properly, should produce a list of 20 fanworks. So this will test how effectively and quickly Ao3MM can return a nested JSON array from 20 fanwork links.	Returns nested JSON array of length=10 populated with data from 10 fanworks.
50 fanwork links	Input first pagination link (i.e. first page of search results for any given fandom tag), for any	Returns nested JSON array of length=10

	fandom with at least 50 fanworks. **Run methods from:** Step1_get_pagination_links.py Step2_get_fanwork_links.py Step3_parse_html_docs.py Step4_merge_json_files.py This will test how effectively and quickly Ao3MM can return a nested JSON array from 50 fanwork links.	populated with data from 10 fanworks; 1 or fewer entries only contains “Unknown” fields (i.e. no metadata was successfully scraped)
100 fanwork links	Input first pagination link (i.e. first page of search results for any given fandom tag), for any fandom with at least 100 fanworks. **Run methods from:** Step1_get_pagination_links.py Step2_get_fanwork_links.py Step3_parse_html_docs.py Step4_merge_json_files.py This will test how effectively and quickly Ao3MM can return a nested JSON array from 50 fanwork links.	Returns nested JSON array of length=10 populated with data from 10 fanworks; 1 or fewer entries only contains “Unknown” fields (i.e. no metadata was successfully scraped)
1,000 fanwork links	Input first pagination link (i.e. first page of search results for any given fandom tag), for any fandom with at least 1,000 fanworks. **Run methods from:** Step1_get_pagination_links.py Step2_get_fanwork_links.py Step3_parse_html_docs.py Step4_merge_json_files.py This will test how effectively and quickly Ao3MM can return a nested JSON array from 50 fanwork links.	Returns nested JSON array of length=10 populated with data from 10 fanworks; 1 or fewer entries only contains “Unknown” fields (i.e. no metadata was successfully scraped)
10,000 fanwork links	Input first pagination link (i.e. first page of search results for any given fandom tag), for any fandom with at least 10,000 fanworks.	Returns nested JSON array of length=10 populated with data

	Run methods from: Step1_get_pagination_links.py Step2_get_fanwork_links.py Step3_parse_html_docs.py Step4_merge_json_files.py This will test how effectively and quickly Ao3MM can return a nested JSON array from 50 fanwork links.	from 10 fanworks; 1 or fewer entries only contains “Unknown” fields (i.e. no metadata was successfully scraped)

Quality Metrics

Similar to performance tests, these quality metrics are defined but not yet measured in Ao3MM. Adding ways to track these metrics is part of the future scope of the project.

Metric	Goal
Time to scrape individual fanwork link and return an HTML document (seconds)	<=60s per fanwork link (to start)
Time to scrape individual pagination link doc and return BeautifulSoup object (seconds)	<=60s per pagination link (to start)
Number of fanwork links successfully scraped before scraping task is interrupted (i.e. terminal / IDE / computer crashes before the task can be completed)	>=20 fanwork links scraped (to start)
Percentage of ‘Unknown’ entries in a nested JSON (where no data is returned / all entries are blank or ‘Unknown’)	<=10% of JSON entries

Alarms & Monitoring

Similar to performance tests, these alarms and monitoring systems are planned but not yet fully defined or implemented in Ao3MM. Adding these alarms and monitoring tasks is part of the future scope of the project.

Alarm	Trigger	Action
scraping task interrupted	process for scraping a list of fanworks does not complete	- Shut down terminal - Check how many fanwork links appended to the fanwork dictionary - Save combined fanwork dictionary as-is - Identify which links were not captured - Rerun scraping task with list of not captured links If number of fanwork links successfully scraped < 20, test in jupyter notebooks using single fanwork link to troubleshoot.
scraping fanwork links is too slow	scraping time (seconds) per individual fanwork increases to > 60 s for 5 or more fanwork links in a row	- Add implicit waits of 10 seconds between each fanwork link to see if this helps processing speed / increase the rate of scraping per fanwork link
greater than 10% unknown entries in final JSON arrays	number of JSON entries with blank / unknown values in combined array reaches > 10% of total fanworks being scraped	- Terminate process - Raise error flag - Test single link in Jupyter notebooks to troubleshoot.

Continuous Testing & Maintenance

- Implement methods to track quality metrics as defined above
- Implement methods to raise alarms as defined above
- Write scripts for performance testing and begin process of incremental performance improvement (identify improvements in processing speed & accuracy based on results of each test before moving to the next performance test)